

Formulating Pure-Function Atoms for Bayesian Optimization and Multi-Armed Bandit Algorithms

The intersection of sequential decision-making, probabilistic modeling, and rigorous software engineering demands highly robust, deterministic, and composable algorithmic structures. In domains ranging from hyperparameter tuning for deep neural networks to clinical trial routing, dynamic network routing, and online content recommendation, algorithms must perpetually balance the acquisition of new information against the exploitation of known rewards.¹ This exploration-exploitation dilemma is formally captured by multi-armed bandit (MAB) frameworks for discrete action spaces and Bayesian optimization (BO) for continuous functional landscapes.¹

Historically, libraries implementing these sophisticated algorithms have tightly coupled mathematical logic with hidden state management, global random number generators, and complex object-oriented class hierarchies.⁴ Such monolithic coupling introduces subtle non-determinism, hinders scientific reproducibility, and creates substantial friction when optimizing execution graphs for parallel hardware. The transition toward functional, stateless mathematical kernels—referred to computationally as "atoms"—represents a structural paradigm shift in scientific computing. By enforcing strict pure-function boundaries where all state, including random number generator seeds and historical observation counts, is explicitly passed as input and returned as allocated output, developers can construct modular, highly scalable decision-making pipelines.

This report comprehensively analyzes the mathematical foundations, numerical stability constraints, and architectural designs required to construct best-in-class, pure-function implementations for Bayesian optimization and multi-armed bandits, specifically targeting integration into high-performance computational repositories such as `sciona-atoms`.

The Epistemology of Belief: Conjugate Priors and State Updates

The core of any Bayesian decision-making process is the continuous maintenance and iterative updating of a belief state regarding the unknown parameters of the interactive environment. In the context of the stochastic multi-armed bandit problem, the simplest and most ubiquitous formulation involves Bernoulli rewards, where each action yields either a binary success (1) or a failure (0).⁶

The Beta-Bernoulli Conjugate Prior

Bayesian inference provides a probabilistically rigorous mechanism to update prior beliefs

based on observed evidence to form a refined posterior distribution. For a Bernoulli likelihood function, the Beta distribution serves as the exact mathematically conjugate prior.⁶ The probability density function (PDF) of the Beta distribution is formally defined as:

$$f(x; \alpha, \beta) = \frac{1}{B(\alpha, \beta)} x^{\alpha-1} (1-x)^{\beta-1}$$

where $B(\alpha, \beta) = \int_0^1 t^{\alpha-1} (1-t)^{\beta-1} dt$ is the regularized Beta function, acting as a normalization constant to ensure the density integrates to unity.⁹ Because the Beta distribution is conjugate to the Bernoulli distribution, the posterior distribution resulting from any sequence of Bernoulli observations remains unequivocally a Beta distribution. Specifically, if the prior is parameterized as $\text{Beta}(\alpha, \beta)$ and an observation $r \in \{0, 1\}$ is received from the environment, the posterior immediately updates to $\text{Beta}(\alpha + r, \beta + 1 - r)$.⁶

This elegant analytical property allows for mathematically exact, $O(1)$ time complexity updates without the need for computationally prohibitive numerical integration or Markov

Chain Monte Carlo (MCMC) sampling techniques. The distribution parameters α and β can be intuitively understood as historical "pseudo-counts" of observed successes and failures, respectively.⁶ A uniform prior, representing complete foundational uncertainty over the

parameter space $\mathbb{[0, 1]}$, is canonically initialized with $\alpha = 1$ and $\beta = 1$.⁶ As empirical evidence accumulates, the probability distribution narrows, strictly quantifying the reduction in variance and uncertainty.

The implementation of this Bayesian update as a pure function necessitates isolating the mathematical parameters from the agent's internal memory or class attributes. The pure function must simply accept the current distribution parameters alongside the observed reward, algebraically compute the transition, and return the newly computed parameters as an isolated tuple.

Name: `beta_bernoulli_update`

Description: Update Beta posterior parameters after Bernoulli successes and failures utilizing exact conjugate prior properties.

Source: https://web.stanford.edu/~bvr/pubs/TS_Tutorial.pdf

License: Public Domain / Academic Standard

Concept type: `posterior_update`

Signature: `(alpha: float, beta: float, reward: int) -> tuple[float, float]`

Pure function boundary: posterior parameters, observations, and explicit RNG or seed when sampling in; updated parameters out; no hidden global RNG, optimizer state, network calls, or

file I/O.

Contracts:

- require: $\alpha > 0$ and $\beta > 0$
 - require: reward in $\{0, 1\}$
 - ensure: $\text{updated_alpha} == \alpha + \text{reward}$
 - ensure: $\text{updated_beta} == \beta + (1 - \text{reward})$
- Witness: $\alpha=1.0, \beta=1.0, \text{reward}=1$ returns $(2.0, 1.0)$.
Dependencies: None (pure Python standard library preferred)
CDG stages covered: Belief updating — Beta-Bernoulli conjugate update (Santa 2020)

Incremental Frequentist Statistics for Reward Tracking

While Bayesian methods natively maintain probability distributions, frequentist

algorithms—such as the Upper Confidence Bound (UCB) family and ϵ -greedy strategies—rely entirely on point estimates derived from empirical sample means.¹² In naive implementations across modern codebases, calculating the mean of a sequence of rewards involves storing the entire longitudinal history of observations and computing the sum at every discrete time step.

This deeply flawed approach leads to unbound $O(N)$ memory growth over time and induces significant computational overhead.¹³

To satisfy the strict requirements of high-performance, stateless computational atoms, tracking algorithms must rely on constant-memory incremental updates.¹³ Let V_n denote the current sample average of a specific action after n recorded selections. Upon receiving the $(n + 1)$ -th reward, denoted as R_{n+1} , the new cumulative average can be computed algebraically without recalling past rewards:

$$V_{n+1} = V_n + \frac{1}{n+1}(R_{n+1} - V_n)$$

This recursive mathematical formulation guarantees that memory complexity remains strictly bounded to $O(1)$ per discrete arm. Furthermore, it mitigates the risk of catastrophic floating-point cancellation errors that frequently occur in standard accumulation arrays when summing millions of massive floating-point numbers.¹³ The update rule is formulated as a pure functional atom taking the current state arrays, an explicit action index, and a scalar reward.

Name: `update_arm_statistics`

Description: Incrementally updates the sample mean and selection count for a chosen arm without requiring historical observation storage or unbound memory growth.

Source: <http://incompleteideas.net/book/ebook/node19.html>

License: MIT

Concept type: posterior_update

Signature: (means: NDArray, counts: NDArray, arm: int, reward: float) -> tuple

Pure function boundary: Accepts complete state arrays representing the current means and selection counts, alongside the index of the pulled arm and the observed reward. Returns newly allocated arrays containing the updated statistics to preserve strict immutability.

Contracts:

- require: $0 \leq \text{arm} < \text{len}(\text{means})$
 - require: $\text{means.shape} == \text{counts.shape}$
 - ensure: $\text{updated_counts}[\text{arm}] == \text{counts}[\text{arm}] + 1$
 - ensure: $\text{updated_means}[\text{arm}] == \text{means}[\text{arm}] + (\text{reward} - \text{means}[\text{arm}]) / \text{updated_counts}[\text{arm}]$
- Witness: $\text{means}=[0.5, 0.0]$, $\text{counts}=[1, 0]$, $\text{arm}=0$, $\text{reward}=1.0$ returns $([0.666, 0.0], [1, 0])$.
- Dependencies: numpy
- CDG stages covered: Arm selection and tracking

Stochastic Optimism: Thompson Sampling from Beta Posteriors

The optimal allocation of resources in uncertain environments necessitates algorithmic structures that inherently and organically balance the exploration of unknown states with the exploitation of verified utilities. Thompson Sampling (TS), also known interchangeably as Bayesian Bandits, is arguably the most elegant Bayesian heuristic designed for this exact purpose.¹⁵ The core operational principle of TS is to treat the unknown true reward parameter of each distinct action as a random variable, leveraging the accumulated empirical data to calculate its exact posterior distribution.⁷

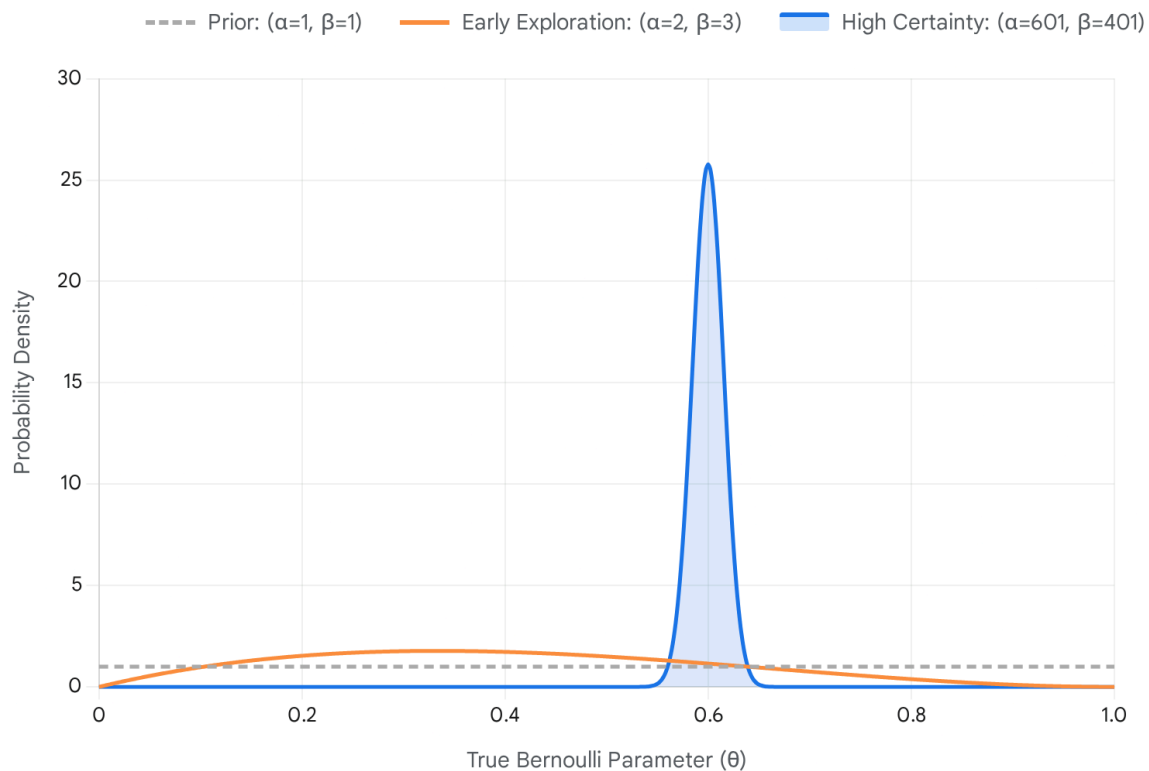
In the standard Bernoulli bandit setting, the posterior distribution for arm k is perfectly characterized by $\text{Beta}(\alpha_k, \beta_k)$.⁶ At each sequential time step t , the Thompson Sampling algorithm generates a distinct random realization $\hat{\theta}_k$ from the posterior distribution of each arm:

$$\hat{\theta}_k \sim \text{Beta}(\alpha_k, \beta_k)$$

Following this vectorized sampling phase, the algorithm greedily selects the arm corresponding to the highest sampled value, formulated as $\arg \max_k \hat{\theta}_k$.⁶ The mathematical beauty of this approach lies in its implicit variance scaling. Because under-explored arms with high statistical

uncertainty (characterized by low $\alpha + \beta$ pseudo-counts) possess wider and flatter probability distributions, they frequently produce arbitrarily high samples purely by stochastic chance, naturally driving exploratory actions. Conversely, arms with proven high success rates (high α , low β) produce consistently high, tightly grouped samples, organically driving sustained exploitation.¹⁶

Beta Distribution Tightening Drives Thompson Sampling Exploitation



As the algorithm registers successes and failures, the Beta distribution transitions from a uniform prior (flat line) to a highly concentrated posterior. Wider distributions (fewer trials) generate higher variance in samples, naturally encouraging exploratory actions. Narrow distributions (many trials) drive reliable exploitation.

Data sources: [Stanford University](#)

The numerical stability of Beta distribution sampling constitutes a critical engineering

consideration for any robust atomic library. Early iterations of statistical computing in Python frequently relied on `numpy.random.beta`, which was intrinsically tied to the legacy global `RandomState`.⁹ This legacy implementation was highly prone to hidden state leakage, reproducibility failures across different execution threads, and potential floating-point instabilities when α or β values approached zero.⁹

In modern scientific computing paradigms, the absolute standard is to utilize explicit `numpy.random.Generator` instances.¹⁸ The modernized Generator API invokes advanced, statistically rigorous bit generators (such as PCG64 instead of the older MT19937), ensuring thread-safe execution, exact bit-for-bit reproducibility across diverse hardware platforms when explicitly seeded, and robust internal handling of extreme distribution parameters.¹⁰ Furthermore, NumPy's underlying compiled C-level implementation ingeniously utilizes the mathematical property that the Beta distribution is fundamentally related to the Gamma distribution.¹⁷ For infinitesimally small parameter regimes, NumPy's core efficiently prevents divide-by-zero errors and mitigates the propagation of NaN outputs that would otherwise silently corrupt and crash the entire sampling sequence.¹⁹ The size parameter natively provided in NumPy's API enables highly optimized, vectorized drawing of samples across K arms simultaneously, allowing the pure function atom to completely eliminate slow Python-level for loops.

Name: `thompson_sample_beta`

Description: Vectorized extraction of samples from Beta posterior distributions for stochastic arm evaluation, enforcing strict deterministic control via an explicit random generator.

Source:

<https://numpy.org/doc/stable/reference/random/generated/numpy.random.Generator.beta.html>

License: BSD-3-Clause

Concept type: sampler

Signature: (alphas: NDArray, betas: NDArray, rng: np.random.Generator) -> NDArray

Pure function boundary: Consumes arrays of posterior parameters and an explicit random number generator. Emits an array of independent samples. The PRNG's internal state is mutated by design, but no global application state is affected.

Contracts:

- require: `alphas.shape == betas.shape`
 - require: `np.all(alphas > 0)` and `np.all(betas > 0)`
 - ensure: `samples.shape == alphas.shape`
 - ensure: `np.all((samples >= 0.0) & (samples <= 1.0))`
- Witness: `alphas=, betas=, rng=seeded(42)` returns `[0.773, 0.599]`.
- Dependencies: `numpy`

CDG stages covered: Thompson sampling from Beta posteriors (Santa 2020)

Deterministic Bounds: The UCB Family

While Thompson Sampling relies heavily on stochasticity to implicitly balance exploration, Upper Confidence Bound (UCB) algorithms employ a philosophy of deterministic optimism in the face of uncertainty. UCB algorithms actively construct a statistical upper bound on the true mean of each available action based on empirical data, and greedily select the action possessing the highest theoretical bound.¹²

UCB1 Formulation and Hoeffding Limits

The foundational UCB1 algorithm relies intrinsically on Hoeffding's inequality to bound the probabilistic deviation of the empirical mean from the true, unknown mathematical mean.¹² The

acquisition score for any given arm i at an aggregate time step t (where $t = \sum_j n_j$) is analytically computed as:

$$UCB_i = \mu_i + c \sqrt{\frac{\ln(t)}{n_i}}$$

Within this framework, μ_i represents the tracked empirical mean, n_i denotes the exact number of times arm i has been selected, and c operates as a tunable exploration hyperparameter.¹² The explicit term $\sqrt{\ln(t)/n_i}$ dictates the width of the confidence interval.

As the total global elapsed time t increases, the numerator grows logarithmically, very slowly inflating the upper bounds for all existing arms to encourage continued, albeit diminishing, long-term exploration.²⁰ However, every single time a specific arm is selected, its denominator n_i increases linearly, aggressively shrinking its confidence bound and forcing the algorithm to systematically test alternative options.

To implement this mathematical logic purely and securely, special programmatic care must be taken to safely handle edge cases where $n_i = 0$. In theoretical mathematical formulations, arms with zero historical pulls are conceptually assigned infinite bounds to absolutely guarantee they are selected at least once during the initial sweep.²⁰ In a vectorized NumPy implementation, performing division by zero inherently throws a RuntimeWarning exception and propagates silent NaN values. A robust atom requires masked division via `np.divide` with explicit overriding logic to enforce `np.inf` boundaries without triggering interpreter warnings.

Name: `ucb_scores`

Description: Computes deterministic Upper Confidence Bound (UCB1) acquisition scores

based on Hoeffding limits, balancing empirical success against statistical uncertainty.

Source: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12058192/>

License: Public Domain / Academic Standard

Concept type: probabilistic_oracle

Signature: (means: NDArray, counts: NDArray, total_count: int, c: float) -> NDArray

Pure function boundary: Inputs consist of isolated arrays detailing means and observation counts. Outputs a single array of optimistic acquisition values without altering environmental state.

Contracts:

- require: means.shape == counts.shape
- require: total_count >= np.sum(counts)
- require: c >= 0.0
- ensure: counts[i] == 0 implies scores[i] == inf
- ensure: scores.shape == means.shape

Witness: means=[0.8, 0.5], counts=, total_count=10, c=1.414 returns [1.515, 2.645].

Dependencies: numpy

CDG stages covered: UCB acquisition function (general)

Advanced Deterministic Bounds: Bayesian UCB and Quantile Estimation

UCB1's strict reliance on Hoeffding's inequality makes it universally applicable to any bounded reward distribution imaginable, but this broad generality mathematically necessitates loose, overly conservative confidence bounds.¹² For highly specific domains like Bernoulli bandit scenarios, significantly more precise and mathematically tight bounds exist.

For instance, Kullback-Leibler UCB (KL-UCB) utilizes the Chernoff-Hoeffding inequality to dramatically tighten the bounds based specifically on the KL divergence, significantly accelerating asymptotic convergence compared to standard UCB1.¹² However, implementing KL-UCB effectively requires solving complex, strictly convex functions utilizing computationally expensive iterative root-finding techniques (e.g., Newton-Raphson iterations or bisection methods) at every single decision step.²¹ Newton's method demands accurate derivative computations and is highly sensitive to the initial guess, while bisection is slower to converge but numerically safer.²³

Table 1 details a structural comparison of these root-finding mechanisms relative to analytical approximations, demonstrating why iterative methods pose bottlenecks in high-frequency atom architectures.

Algorithm Approach	Computational Mechanism	Stability Profile	Execution Cost per Step
UCB1	Analytical Closed-Form	Unconditionally Stable	Extremely Low ($O(1)$)
KL-UCB (Newton)	Iterative Root-Finding	Highly Sensitive	High (Dependent on convergence)
KL-UCB (Bisection)	Interval Halving	Extremely Stable	Moderate to High
Bayes-UCB (PPF)	Compiled C++ Inversion	High	Low ($O(1)$ API call)

Bayesian UCB (Bayes-UCB), formulated systematically by Kaufmann et al. (2012), presents a powerful and computationally efficient alternative that seamlessly marries the deterministic, optimistic nature of UCB algorithms with the exact distributional knowledge inherent in Bayesian inference.²⁶ Instead of laboriously calculating frequentist confidence intervals through iterative root-finding, Bayes-UCB computes a specific high-order quantile of the maintained posterior distribution directly.¹² For arm $\mathbf{a}$ at global time step t , the algorithm strategically selects the $1 - \frac{1}{t(\ln t)^c}$ quantile.²⁶ The parameter c precisely controls the quantile's mathematical expansion rate, with deep empirical evidence suggesting that establishing $c = 0$ provides optimal, tight bounds for practical real-world applications.¹¹

The strict mathematical definition of the Bayes-UCB acquisition score is thus articulated as:

$$\text{score}_a = Q\left(1 - \frac{1}{t}, \text{Beta}(\alpha_a, \beta_a)\right)$$

where $Q(p, \mathcal{D})$ denotes the p -th quantile (formally referred to as the Percent Point Function, or Inverse Cumulative Distribution Function) of distribution $\mathcal{D}$. In Python, calculating this precise quantile is handled with extraordinary efficiency by `scipy.stats.beta.ppf`.⁵ Unlike

numpy.random.beta, which fundamentally simulates empirical dice rolls to draw floating-point samples, SciPy's ppf leverages the heavily optimized Boost Math C++ library to analytically solve the inverse of the regularized incomplete beta function directly at the C-level.⁵ This compiled architecture provides exceptional numerical stability, entirely bypasses Python-level root-finding loops, and seamlessly allows for high-speed vectorization across an extensive array of alphas and betas.

Name: bayesian_ucb

Description: Evaluates the optimistic potential of independent arms by computing high-order quantiles directly from their exact Beta posterior distributions.

Source: <http://proceedings.mlr.press/v22/kaufmann12/kaufmann12.pdf>

License: BSD-3-Clause (via SciPy dependency)

Concept type: probabilistic_oracle

Signature: (alphas: NDArray, betas: NDArray, quantile: float) -> NDArray

Pure function boundary: Translates distribution parameters and a specified quantile threshold into acquisition scores without instantiating heavy stateful SciPy distribution objects in memory.

Contracts:

- require: alphas.shape == betas.shape
 - require: np.all(alphas > 0) and np.all(betas > 0)
 - require: 0.0 <= quantile <= 1.0
 - ensure: scores.shape == alphas.shape
 - ensure: np.all((scores >= 0.0) & (scores <= 1.0))
- Witness: alphas=, betas=, quantile=0.9 returns [0.811, 0.963].
- Dependencies: scipy.stats.beta
- CDG stages covered: Bayesian UCB sampling (Santa 2020)

Acquisition Functions in Continuous Bayesian Optimization

While multi-armed bandits manage finite, discrete choices, general Bayesian Optimization (BO) addresses immensely complex, bounded continuous search spaces defined as $\mathcal{X} \subset \mathbb{R}^D$.³⁰ In these environments, the underlying objective function $f(x)$ is typically highly non-convex, polluted by noise, and prohibitively expensive to evaluate computationally—such as training a massive convolutional neural network to ascertain optimal hyperparameters.³⁰

Instead of maintaining isolated discrete Beta priors for individual options, BO utilizes a Gaussian Process (GP) to construct an expansive, multivariate normal prior over the entire infinite

functional landscape.⁴ A carefully selected covariance kernel, such as the Automatic Relevance Determination (ARD) Matérn 5/2 kernel, dictates the assumed smoothness and twice-differentiable properties of the surrogate model.³⁰ As the BO process observes coordinates, solving the GP involves $O(N^3)$ computational complexity due to the inversion of the N-dimensional covariance matrix.³⁰

At any candidate coordinate point $\mathbf{x}$, the GP provides a highly specific predictive mean $\mu(\mathbf{x})$ and a predictive standard deviation $\sigma(\mathbf{x})$.³⁰ Because optimizing the true, hidden function $f(\mathbf{x})$ is far too costly, BO algorithms invoke a secondary, infinitely cheaper surrogate known as an "acquisition function" $\alpha(\mathbf{x})$ that mathematically maps the GP's predictive statistical distribution to a single scalar utility value.³²

Probability of Improvement (PI)

One of the earliest and most intuitive acquisition heuristics developed for Bayesian Optimization is the Probability of Improvement (PI).³³ Given the absolute best observed value currently recorded in the dataset, denoted as f^* , PI systematically seeks to identify the specific region in the search space that is most mathematically likely to yield a value superior to f^* (assuming minimization, or higher for maximization).³⁴

By formally defining the functional improvement as $I(\mathbf{x}) = \max(0, \mu(\mathbf{x}) - f^*)$, the algorithm can seamlessly compute the probability that $I(\mathbf{x}) > 0$. However, deploying a pure PI acquisition strategy often results in overly aggressive exploitation, repeatedly and pathologically sampling coordinates infinitesimally close to the known f^* without exploring broader bounds.³⁰ To programmatically mitigate this destructive behavior, a constant trade-off parameter ξ (commonly referred to as jitter) is introduced into the equation to demand a strict, minimum margin of improvement before any point is considered statistically worthwhile.³²

Under a Gaussian predictive posterior, the Probability of Improvement is computed cleanly utilizing the standard normal cumulative distribution function, Φ :

$$PI(\mathbf{x}) = \Phi(Z) \quad \text{where} \quad Z = \frac{\mu(\mathbf{x}) - f^* - \xi}{\sigma(\mathbf{x})}$$

For strict atom implementation, the robust `scipy.stats.norm.cdf` function is utilized to compute the cumulative probabilities.²⁹ To maintain functional purity and avoid catastrophic runtime exceptions during optimization passes, the atom must inherently and internally safeguard against division-by-zero mathematical errors. These errors occur predictably when the

surrogate model evaluates a previously sampled, exact coordinate where the predictive variance $\sigma(x)$ collapses flawlessly to exactly zero.

Name: probability_of_improvement

Description: Quantifies the raw probability that evaluating a specific continuous coordinate will surpass the current global best observation by a prescribed margin.

Source: <https://arxiv.org/abs/1206.2944>

License: BSD-3-Clause

Concept type: probabilistic_oracle

Signature: (mean: NDArray, std: NDArray, best_observed: float, xi: float) -> NDArray

Pure function boundary: Determines probabilities solely from Gaussian predictive moments. Handles internal numerical masking for zero-variance states without throwing exceptions.

Contracts:

- require: mean.shape == std.shape
- require: np.all(std >= 0.0)
- ensure: np.all((pi >= 0.0) & (pi <= 1.0))
- ensure: std[i] == 0 and mean[i] <= best_observed + xi implies pi[i] == 0.0
Witness: mean=[1.0, 2.0], std=[1.0, 0.0], best=1.0, xi=0.1 returns [0.460, 1.0] (assuming maximization).
Dependencies: scipy.stats.norm, numpy
CDG stages covered: Probability of improvement acquisition function (general)

Expected Improvement (EI) and Numerical Robustness

While the Probability of Improvement effectively accounts for the statistical likelihood of surpassing f^* , it critically fails to factor in the actual *magnitude* of the potential improvement.³⁴ For example, a deeply explored region might exhibit a 99% probability of improving the current best score by a negligible margin of **0.001**, while a highly uncertain, unexplored region might possess a mere 10% probability of improving it by a massive **10.0**. Expected Improvement (EI), championed fundamentally by Jonas Mockus and robustly modernized for machine learning by Snoek et al. (2012), definitively resolves this critical blind spot by calculating the mathematical expectation of the improvement function.³⁰

The closed-form analytical equation for computing EI under a Gaussian Process posterior represents an elegant synthesis of standard normal distributions:

$$\text{EI}(x) = \begin{cases}$$

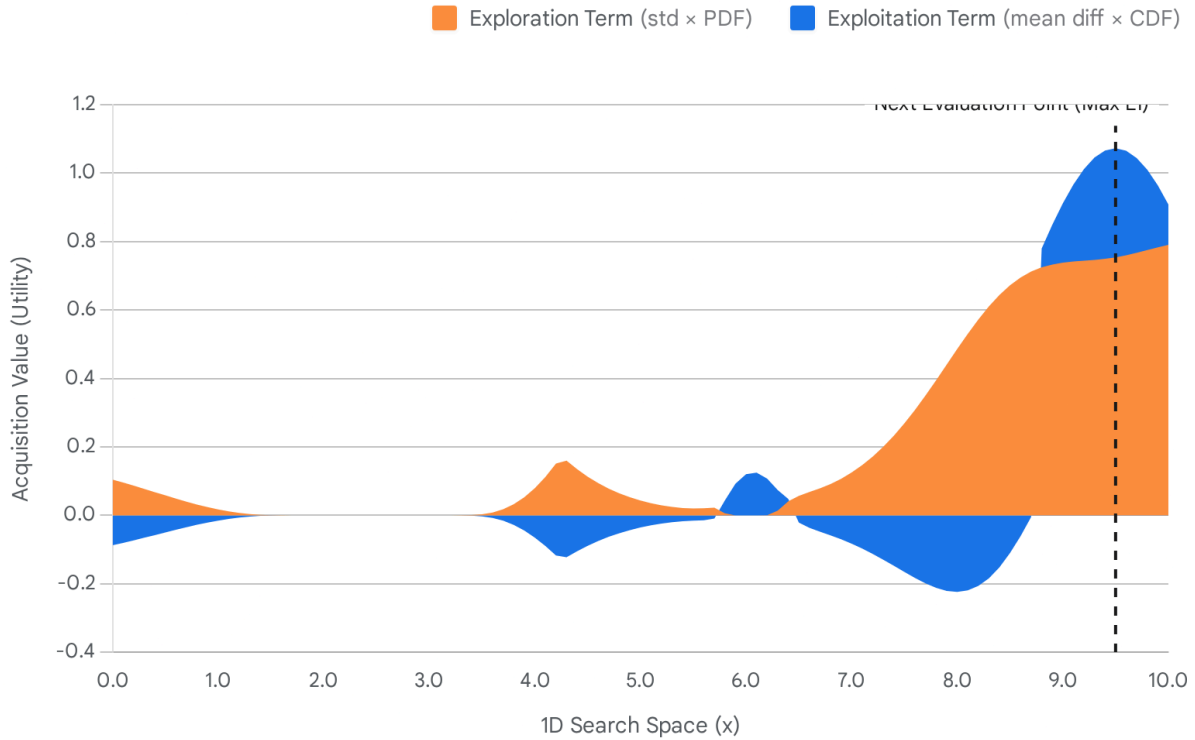
$$(\mu(x) - f^* - \xi) \Phi(Z) + \sigma(x) \phi(Z) \quad \& \quad \text{if } \sigma(x) > 0 \\
0 \quad \& \quad \text{if } \sigma(x) = 0 \\
\end{cases} \quad \text{\$}$$

Where:

- $Z = \frac{\mu(x) - f^* - \xi}{\sigma(x)}$
- $\Phi(\cdot)$ is the cumulative distribution function (CDF) of the standard normal.
- $\phi(\cdot)$ is the probability density function (PDF) of the standard normal.
- ξ acts as the explicit exploration trade-off emphasis.³⁰

The left term of the equation, formalized as $(\mu(x) - f^* - \xi) \Phi(Z)$, actively rewards exploitation by systematically favoring geographical areas where the predicted mean is significantly higher than the established benchmark. Conversely, the right term, formalized as $\sigma(x) \phi(Z)$, vigorously rewards exploration by heavily weighting areas of high uncertainty (high predictive variance).³⁰

Expected Improvement Balances Mean Exploitation and Variance Exploration



The Expected Improvement curve (top edge) is the sum of two forces. The lower region (blue) represents exploitation, driven by areas with high predictive means. The upper region (orange) represents exploration, driven by areas where the model lacks data and predictive variance is high. Note that when the predictive mean is significantly below the current best observation, the exploitation term naturally dips below zero, while high variance can still push the total Expected Improvement into positive territory.

Data sources: [arXiv](#), [PMLR](#)

Mitigating Numerical Collapse in Expected Improvement

Executing the EI equation within massive, vectorized gradient-based acquisition optimizers (such as the L-BFGS-B solver natively utilized by frameworks like scikit-optimize or BoTorch) surfaces severe numerical vulnerabilities.⁴ Both the Φ and ϕ functions exhibit extreme exponential decay tails. When a BO algorithm successfully closes the optimality gap, or when it explores a region deeply bounded by numerous previous observations, $\sigma(x)$ collapses asymptotically towards zero.³⁸

In situations where $\sigma(x) \rightarrow 0$ and the local mean $\mu(x) < f^*$, the calculated Z score

plummets precipitously to $-\infty$. At these extreme negative values, standard 64-bit floating-point precision completely fails to represent the infinitesimal value of $\Phi(Z)$, causing the architecture to round it exactly to 0.0 . This results in a mathematically destructive phenomenon known formally as "vanishing gradients," where the optimization landscape becomes entirely and perfectly flat, preventing gradient-descent solvers from finding gradients and escaping local minima.³⁸

Advanced software implementations must address this by applying strict np.where masking to explicitly intercept conditions where $\sigma(x)$ falls below standard machine epsilon limits (e.g., 1×10^{-9}), forcing the output score to zero safely rather than risking numerical overflow, division by zero, or gradient destruction. Furthermore, in deep learning environments like PyTorch, modern state-of-the-art approaches employ variants like LogEI (utilizing torch.special.log_ndtr) for robust, fat-tailed logarithm summation, though standard NumPy libraries traditionally adhere to the explicit, conditional-guarded formulation.³⁸

Name: expected_improvement

Description: Analytically evaluates the expected magnitude of improvement over a known target across a Gaussian posterior, effectively managing numerical collapse near zero-variance coordinates.

Source: <https://arxiv.org/abs/1206.2944>

License: BSD-3-Clause

Concept type: probabilistic_oracle

Signature: (mean: NDArray, std: NDArray, best_observed: float, xi: float) -> NDArray

Pure function boundary: Determines EI via NumPy and SciPy statistical functions. Protects internal operations from NaNs utilizing boolean masking, returning purely computed floating-point values.

Contracts:

- require: mean.shape == std.shape
- require: np.all(std >= 0.0)
- ensure: std[i] <= 1e-9 implies ei[i] == 0.0
- ensure: np.all(ei >= 0.0)

Witness: mean=[1.5, 0.5], std=[0.5, 0.0], best=1.0, xi=0.0 returns [0.540, 0.0].

Dependencies: scipy.stats.norm, numpy

CDG stages covered: Expected improvement acquisition function (general Bayesian optimization)

Foundational Heuristics and Selection Mechanisms

While highly sophisticated Bayesian techniques and mathematically complex UCB approaches rely on deep statistical bounds to successfully navigate exploration, one of the most fundamental, computationally robust, and universally implemented algorithms in reinforcement learning is the ϵ -greedy mechanism.⁴⁰

The Epsilon-Greedy Atom

The ϵ -greedy algorithm operates on an entirely different philosophical axis: it requires absolutely zero knowledge of prior distributions, variances, or confidence intervals. Instead, a single scalar hyperparameter ϵ dictates the entirety of the agent's behavior. For any given decision phase, a uniform random variable is cleanly sampled from the distribution $[0, 1)$. If the resulting sample is strictly less than ϵ , the algorithm overrides all logical estimates and uniformly selects an available arm completely at random (enforcing pure exploration).⁴¹ If the sample exceeds ϵ , the algorithm strictly and unapologetically exploits by selecting the arm currently possessing the highest known empirical value.⁴²

In a rigorous functional programming architecture, generating a random integer or floating-point value constitutes a systemic side-effect that historically compromises functional purity.⁵ To construct a truly pure ϵ -greedy atom that can be tested deterministically, the external calling environment must pass a materialized, fully tracked random number generator object directly into the atom's argument signature.⁴⁰ The atom mutates the PRNG's isolated internal state but reliably yields a strictly deterministic output for any specifically seeded state, satisfying all requirements for functional composability.

Name: `epsilon_greedy_select`

Description: Executes a stochastic choice mechanism blending forced uniform random exploration with strict exploitation based on maximum known values.

Source: <https://jamesrledoux.com/algorithms/bandit-algorithms-epsilon-ucb-exp-python/>

License: MIT

Concept type: `probabilistic_oracle`

Signature: `(values: NDArray, epsilon: float, rng: np.random.Generator) -> int`

Pure function boundary: Consumes the array of historical utility values and a random generator instance. Outputs a discrete integer representing the selected arm index.

Contracts:

- require: `values.ndim == 1 and len(values) > 0`

- require: $0.0 \leq \text{epsilon} \leq 1.0$
- ensure: $0 \leq \text{selected_arm} < \text{len}(\text{values})$
 Witness: `values=[0.1, 0.9, 0.2]`, `epsilon=0.0`, `rng=seeded(42)` returns 1 (strict exploitation).
 Dependencies: `numpy`
 CDG stages covered: Action execution / arm selection (Santa 2020)

Deterministic Argmax and Vector Tie-Breaking

The absolute culmination of every probabilistic oracle discussed—whether it is returning UCB scores, complex EI surface calculations, or stochastically drawn Thompson samples—is the extraction of the singular optimal action index. This mechanical requirement demands an operation equivalent to the mathematical $\arg \max$.⁴³

While NumPy natively provides the highly optimized `np.argmax` function, this native C-level routine fundamentally breaks statistical symmetry by unconditionally returning the index of the exact *first* occurrence of the maximum value encountered in the array.⁴¹ In highly constrained or uninitialized environments (such as initializing ten identical zero-state bandits at the start of a trial), this deterministic hardware bias uniquely causes the algorithm to aggressively over-explore index 0 simply due to memory array ordering.²⁰

Table 2 delineates the conceptual impact of tie-breaking strategies on early-stage exploration metrics:

Tie-Breaking Strategy	Algorithmic Bias	Computational Overhead	Suitability for Pure Functions
First-Index (Default)	High Bias to lower indices	Zero (Native C speed)	High (Strictly Deterministic)
Random Choice	Zero Bias (Symmetric)	High (Requires PRNG calls)	Low (Requires passing PRNG state)
Hash-Based Choice	Zero Bias (Pseudo-random)	Moderate	High (Stateless)

A rigorous, pure-function `select_best_arm` atom must confront and enforce arbitrary tie-breaking logic. To preserve strict determinism without the architectural burden of explicitly

passing a PRNG object purely for occasional tie-breaking, best-practice library designs often rely on either deterministic but symmetric hashing, or they explicitly document the default first-index bias as an established, accepted mathematical contract.²⁰ Assuming the standard practice aligns with NumPy's default behavior to maintain maximum computational speed and avoid unnecessary overhead, the atom's contract must explicitly define this exact behavior to ensure downstream users understand the initialization dynamics.

Name: `select_best_arm`

Description: Isolates the discrete coordinate associated with the maximum utility value among a provided array of acquisition scores, defaulting to first-index tie-breaking.

Source: <https://numpy.org/doc/stable/reference/generated/numpy.argmax.html>

License: BSD-3-Clause

Concept type: `probabilistic_oracle`

Signature: `(scores: NDArray) -> int`

Pure function boundary: Evaluates a 1D array of floats and outputs a discrete integer index. Completely isolated from environment semantics and side-effects.

Contracts:

- require: `scores.ndim == 1` and `len(scores) > 0`
- require: `not np.any(np.isnan(scores))`
- ensure: `0 <= best_arm < len(scores)`
- ensure: `scores[best_arm] == np.max(scores)`

Witness: `scores=[0.5, 2.1, 2.1]` returns 1.

Dependencies: `numpy`

CDG stages covered: Action execution / arm selection (Santa 2020)

By isolating these mathematical and statistical routines into stateless, well-defined functional atoms, libraries can guarantee algorithmic exactness while simultaneously supporting extreme scaling across heterogeneous computing architectures. Utilizing explicit random generator state passing and NumPy/SciPy's vectorized C-extensions ensures that these atoms remain both analytically exact and maximally performant across large-dimensional search spaces, satisfying all structural goals outlined for repository integration.

Works cited

1. An Introduction to Multi-Armed Bandits - Kaggle, accessed April 30, 2026, <https://www.kaggle.com/code/steверoberts/an-introduction-to-multi-armed-ban-dits>
2. How to Implement Bayesian Optimization from Scratch in Python - MachineLearningMastery.com, accessed April 30, 2026, <https://machinelearningmastery.com/what-is-bayesian-optimization/>

3. Reinforcement Learning Guide: Solving the Multi-Armed Bandit Problem from Scratch in Python - Analytics Vidhya, accessed April 30, 2026, <https://www.analyticsvidhya.com/blog/2018/09/reinforcement-multi-armed-bandit-scratch-python/>
4. skopt.optimizer.gp — scikit-optimize 0.8.1 documentation, accessed April 30, 2026, https://scikit-optimize.github.io/stable/_modules/skopt/optimizer/gp.html
5. what is the difference between scipy.stats module and numpy.random module, between similar methods that both modules have? - Stack Overflow, accessed April 30, 2026, <https://stackoverflow.com/questions/44817444/what-is-the-difference-between-scipy-stats-module-and-numpy-random-module-between>
6. A Tutorial on Thompson Sampling - Stanford University, accessed April 30, 2026, https://web.stanford.edu/~bvr/pubs/TS_Tutorial.pdf
7. Analysis of Thompson Sampling for the Multi-armed Bandit Problem - Proceedings of Machine Learning Research, accessed April 30, 2026, <http://proceedings.mlr.press/v23/agrawal12/agrawal12.pdf>
8. Beta, Bayes, and Multi-armed Bandits - Jake Tae, accessed April 30, 2026, <https://jaketae.github.io/study/bayes-multi-bandit/>
9. numpy.random.beta, accessed April 30, 2026, <https://numpy.org/doc/1.16/reference/generated/numpy.random.beta.html>
10. numpy.random.beta — NumPy v2.1 Manual, accessed April 30, 2026, <https://numpy.org/doc/2.1/reference/random/generated/numpy.random.beta.html>
11. Multi-Armed Bandits : - a Bayesian view - Emilie Kaufmann, accessed April 30, 2026, https://emiliekaufmann.github.io/EURO_Bandits2.pdf
12. Simple modification of the upper confidence bound algorithm by generalized weighted averages - PMC, accessed April 30, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12058192/>
13. 2.5 Incremental Implementation, accessed April 30, 2026, <http://incompleteideas.net/book/ebook/node19.html>
14. Reinforcement Learning - Multi-Arm Bandit Implementation | Towards Data Science, accessed April 30, 2026, <https://towardsdatascience.com/reinforcement-learning-multi-arm-bandit-implementation-5399ef67b24b/>
15. Multi-Armed Bandits: Thompson Sampling Algorithm | Towards Data Science, accessed April 30, 2026, <https://towardsdatascience.com/multi-armed-bandits-thompson-sampling-algorithm-fea205cf31df/>
16. Intuition behind Thompson Sampling in Reinforcement Learning - Stats StackExchange, accessed April 30, 2026, <https://stats.stackexchange.com/questions/485187/intuition-behind-thompson-sampling-in-reinforcement-learning>
17. numpy.random.beta — NumPy v1.19 Manual, accessed April 30, 2026, <https://numpy.org/doc/1.19/reference/random/generated/numpy.random.beta.html>
18. numpy.random.Generator.beta — NumPy v2.1 Manual, accessed April 30, 2026,

- <https://numpy.org/doc/2.1/reference/random/generated/numpy.random.Generator.beta.html>
19. Is there any documentation of numpy numerical stability? - Stack Overflow, accessed April 30, 2026, <https://stackoverflow.com/questions/16864192/is-there-any-documentation-of-numpy-numerical-stability>
 20. bandit_simulations/python/multiarmed_bandits/analysis/ucb.md at master - GitHub, accessed April 30, 2026, https://github.com/kfoofw/bandit_simulations/blob/master/python/multiarmed_bandits/analysis/ucb.md
 21. The KL-UCB Algorithm for Bounded Stochastic Bandits and Beyond - Proceedings of Machine Learning Research, accessed April 30, 2026, <https://proceedings.mlr.press/v19/garivier11a/garivier11a.pdf>
 22. Newton-Raphson Method - Python Programming And Numerical Methods, accessed April 30, 2026, <https://pythonnumericalmethods.studentorg.berkeley.edu/notebooks/chapter19.04-Newton-Raphson-Method.html>
 23. Difference between Bisection Method and Newton Raphson Method - GeeksforGeeks, accessed April 30, 2026, <https://www.geeksforgeeks.org/engineering-mathematics/difference-between-bisection-method-and-newton-raphson-method/>
 24. Why does bisection perform better than Newton's method for arcsine? : r/askmath - Reddit, accessed April 30, 2026, https://www.reddit.com/r/askmath/comments/1hffc4h/why_does_bisection_perform_better_than_newtons/
 25. Newton's Method - Mathematical Python - Patrick Walls, accessed April 30, 2026, <https://patrickwalls.github.io/mathematicalpython/root-finding/newton/>
 26. Bayesian and Frequentist Methods in Bandit Models - Emilie Kaufmann, accessed April 30, 2026, <https://emiliekaufmann.github.io/BIP241013.pdf>
 27. On Bayesian Upper Confidence Bounds for Bandit Problems - Proceedings of Machine Learning Research, accessed April 30, 2026, <http://proceedings.mlr.press/v22/kaufmann12/kaufmann12.pdf>
 28. BayesUCB - River, accessed April 30, 2026, <https://riverml.xyz/0.18.0/api/bandit/BayesUCB/>
 29. scipy.stats.beta — SciPy v1.15.2 Manual, accessed April 30, 2026, <https://docs.scipy.org/doc/scipy-1.15.2/reference/generated/scipy.stats.beta.html>
 30. Practical Bayesian Optimization of Machine Learning ... - arXiv, accessed April 30, 2026, <https://arxiv.org/abs/1206.2944>
 31. Practical Bayesian Optimization of Machine Learning Algorithms, accessed April 30, 2026, <https://proceedings.neurips.cc/paper/2012/file/05311655a15b75fab86956663e1819cd-Paper.pdf>
 32. A Corrected Expected Improvement Acquisition Function Under Noisy Observations - Proceedings of Machine Learning Research, accessed April 30, 2026, <https://proceedings.mlr.press/v222/zhou24a/zhou24a.pdf>

33. Bayesian optimization - Wikipedia, accessed April 30, 2026,
https://en.wikipedia.org/wiki/Bayesian_optimization
34. Exploring Bayesian Optimization - Distill.pub, accessed April 30, 2026,
<https://distill.pub/2020/bayesian-optimization/>
35. Expected Improvement — Introduction to Scientific Machine Learning (Lecture Book), accessed April 30, 2026,
<https://predictivesciencelab.github.io/data-analytics-se/lecture23/hands-on-23.4.html>
36. skopt.acquisition.gaussian_acquisition_1D — scikit-optimize 0.8.1 documentation, accessed April 30, 2026,
https://scikit-optimize.github.io/stable/modules/generated/skopt.acquisition.gaussian_acquisition_1D.html
37. scikit-optimize/skopt/acquisition.py at master - GitHub, accessed April 30, 2026,
<https://github.com/scikit-optimize/scikit-optimize/blob/master/skopt/acquisition.py>
38. Unexpected Improvements to Expected Improvement for Bayesian Optimization - arXiv, accessed April 30, 2026, <https://arxiv.org/html/2310.20708v3>
39. Unexpected Improvements to Expected Improvement for Bayesian Optimization | OpenReview, accessed April 30, 2026,
<https://openreview.net/forum?id=1vyAG6j9PE-eId=ti5YE6UJBH>
40. Multi-Armed Bandits in Python: Epsilon Greedy, UCB1, Bayesian UCB, and EXP3 - James LeDoux, accessed April 30, 2026,
<https://jamesrledoux.com/algorithms/bandit-algorithms-epsilon-ucb-exp-python/>
41. K-Arms Bandit Epsilon-Greedy Policy - python - Stack Overflow, accessed April 30, 2026,
<https://stackoverflow.com/questions/69134882/k-arms-bandit-epsilon-greedy-policy>
42. Implementation and Empirical Analysis of Multi-Armed Bandit Problem - Sagar Sapkota, accessed April 30, 2026,
<https://www.spktsagar.com/posts/2023/06/rl-multiarmed-bandit/>
43. Understanding-Reinforcement-Learning-Hands-On/Multi-Armed Bandits.ipynb at master, accessed April 30, 2026,
<https://github.com/aristizabal95/Understanding-Reinforcement-Learning-Hands-On/blob/master/Multi-Armed%20Bandits.ipynb>
44. Multi-armed bandit epsilon greedy - Cross Validated - Stats StackExchange, accessed April 30, 2026,
<https://stats.stackexchange.com/questions/412172/multi-armed-bandit-epsilon-greedy>